

GMCP-R：面向断续通信的记忆链可验证历史状态恢复协议

GMCP-R: A Memory-Chain-Based Protocol for Verifiable Historical State Recovery in Intermittent Communications

Abstract (摘要)

在断续通信场景中，通信双方即使能够在断连或攻击检测后重新建立连接，也仍需验证恢复后的消息是否接续在可信历史状态之后。针对恢复后历史连续性难以验证的问题，本文提出 GMCP-R (Memory-Continuity-Aware Communication Protocol with Recovery)，一种面向断续通信的记忆链安全恢复协议。GMCP-R 通过 memory chain 将消息内容、序列号、会话上下文和前序记忆状态绑定为连续演化的历史状态链，并结合 MemoryTicket 与 Checkpoint 对恢复点、记忆状态和最近可信快照进行认证与约束，从而实现可验证的状态恢复。基于 Python TCP 原型的实验结果表明，在测试攻击场景下，GMCP-R 达到 100% 攻击检测率、0% 误接受率和 0% 误拒绝率；在攻击与断连恢复场景下，恢复成功率和恢复后记忆一致率均达到 100%；五类无效 MemoryTicket 均被拒绝。性能实验显示，GMCP-R 的本地纯计算吞吐量均值为 65,931 msg/s，并在 1 至 20 个并发客户端下保持 100% 成功率。结果表明，GMCP-R 能够以可接受的计算开销，为断续通信提供历史连续性验证与安全状态恢复支持。

Keywords (关键词)：安全通信；历史连续性；状态恢复；断续通信；MemoryTicket；Checkpoint；攻击检测；通信协议

1. Introduction (引言)

1.1 研究背景

随着远程设备控制、工业现场通信、边缘 IoT 数据采集和安全日志同步等应用的发展，通信系统越来越多地运行在不稳定链路和间歇连接环境中。链路中断、短时丢包、设备切换、网络抖动以及攻击检测后的主动断开，都会使通信双方进入恢复过程。现有 TLS 1.3、QUIC 等协议已经能够支持连接重建、密钥上

下文恢复或会话快速恢复，但这类机制主要解决的是“如何重新建立可通信信道”的问题，并不直接保证恢复后的消息序列仍然接续在可信的历史状态之后。

在许多安全敏感场景中，恢复后的通信状态是否可信与连接是否重新建立同样重要。例如，在远程控制、工业遥测、交易流水同步、医疗数据同步和审计日志传输中，通信双方不仅需要继续发送新消息，还需要确认恢复点之前的消息历史没有被丢弃、篡改、重放或回滚。若攻击者在断连或恢复过程中诱导通信双方接受旧状态、跨会话状态或伪造状态，即使连接已经恢复，后续通信也可能建立在不一致或不可信的历史基础之上。

因此，断续通信中的安全恢复不应只关注连接恢复，还应关注历史状态恢复。也就是说，恢复机制需要回答两个问题：第一，恢复后的消息应从哪个历史位置继续；第二，该恢复位置对应的历史状态是否仍然可信。本文围绕这一问题展开研究，提出一种基于记忆链的安全恢复协议 GMCP-R，使通信双方能够在攻击或断连后恢复并验证由消息序列和记忆状态共同刻画的历史通信状态。

1.2 问题定义：可验证历史状态恢复

本文关注断续通信中的可验证历史状态恢复问题。设通信双方在第 i 条消息附近发生异常中断、攻击检测或主动断开。此时，恢复机制不仅需要使通信双方重新进入可通信状态，还需要确认恢复后的消息序列是否能够接续在此前已接受的合法历史之后。换言之，恢复过程需要验证恢复点、恢复状态和最近可信快照之间的一致关系，避免攻击者通过旧状态、跨会话状态、回滚状态或伪造状态诱导双方继续通信。

在本文中，恢复点由 `last_seq` 表示，表示通信双方共同接受的最新 DATA 报文序号；恢复状态由 `last_mem` 表示，表示截至该序号处理后形成的链式历史记忆状态；恢复锚点由 Checkpoint 表示，包含最近可信快照 `{ckpt_seq, ckpt_mem}`，用于缩短恢复时需要验证或重放的历史窗口。一次安全恢复并不是简单重连，而是要求 `last_seq`、`last_mem` 与 Checkpoint 在 MemoryTicket 认证和状态单调性约束下能够对应同一条合法消息历史。若攻击者尝试使用过期票据、重复票据、跨会话票据、回滚票据或篡改后的记忆状态发起恢复，服务端应拒绝该恢复请求，并避免更新本地通信状态。

表 1. 现有方案与 GMCP-R 的能力对比

方法	连接/会话恢复	历史验证	历史状态恢复	主要局限
TLS/QUIC 会话恢复	支持	不直接支持	恢复加密或会话上下文	不验证断连期间应用层历史连续性
Seq+MAC	不直接支持	部分支持	不支持	缺少记忆链，难以检测 prev_mem 伪造或

方法	连接/会话恢复	历史验证	历史状态恢复	主要局限
				历史状态回滚
Hash Chain	不直接支持	支持	依赖历史重放	长会话恢复开销随历史长度线性增长
Ticket Only	支持	不支持	恢复会话票据	缺少消息内容、序列号与历史记忆状态绑定
GMCP-R	依赖底层重连	支持	支持可验证历史状态恢复	以额外哈希、HMAC和票据管理开销换取可验证恢复

1.3 Research Gap and Challenges（研究空白与挑战）

现有轻量认证与恢复方案各自覆盖了断续通信安全问题的一部分，但尚缺少一种面向恢复后历史状态可信性的统一机制。Seq+MAC 主要解决单包完整性与序列保护，难以表达跨消息历史状态；Ticket Only 能支持连接或会话恢复，却不验证恢复点之前的消息历史；Hash Chain 可以验证历史篡改，但单独使用时缺少安全恢复票据和低成本恢复路径。因此，断续通信中的可验证历史状态恢复仍面临以下挑战。

C1. 连接恢复与历史状态恢复解耦。

现有会话恢复机制可以帮助通信双方重新建立连接、恢复密钥上下文或降低握手延迟，但不能直接证明恢复后的消息序列仍然接续在可信历史状态之后。因此，协议需要在连接恢复之外维护可验证的历史状态。

C2. 恢复过程容易受到重放、回滚和跨会话攻击。

攻击者可能在断连或恢复阶段提交旧票据、重复票据、跨会话票据或被篡改的恢复状态，使通信双方错误地从旧位置或错误历史继续通信。因此，恢复凭证必须同时具备认证完整性、时效性、一次性和状态单调性约束。

C3. 纯 Hash Chain 的恢复成本随历史长度增长。

Hash Chain 能够提供历史绑定能力，但如果恢复时需要从会话初始点或较早状态重放全部历史消息，则长会话场景下恢复开销会随历史长度线性增长。断续通信协议需要在保持历史验证能力的同时，将恢复成本限制在可接受范围内。

C4. 协议需要在受限环境中保持可部署性。

断续通信、边缘 IoT、远程设备和工业现场网络通常不能依赖复杂可信硬件或完整历史日志存储。协议设计需要在安全恢复能力、计算开销、状态存储和实现复杂度之间取得平衡。

1.4 本文贡献

本文主要贡献如下。

(1) **提出断续通信中的可验证历史状态恢复问题。**本文区分连接恢复、消息完整性验证与历史状态恢复，明确恢复点 `last_seq`、恢复状态 `last_mem` 与恢复锚点 Checkpoint 的语义，指出安全恢复不仅要使通信双方重新进入可通信状态，还需要验证恢复后的消息是否接续在可信历史状态之后。

(2) **设计基于 `memory chain` 的历史连续性验证机制。**GMCP-R 将会话标识、时代号、序列号、载荷哈希、发送方标识和前序记忆状态绑定到逐消息演化的记忆链中，使接收端能够通过 `prev_mem` 与本地 `last_mem` 的一致性检查检测历史断裂、旧消息重放和 `prev_mem` 伪造。

(3) **提出 MemoryTicket 与 Checkpoint 结合的安全状态恢复机制。**MemoryTicket 以服务器认证票据绑定 `last_seq`、`last_mem`、`checkpoint`、过期时间、`nonce` 和会话上下文，支持对恢复状态的认证、时效性检查、一次性检查、回滚检测和跨会话绑定；Checkpoint 用于限制长会话恢复时的历史重放范围，降低纯 Hash Chain 恢复开销。

(4) **实现原型并系统评估 GMCP-R 的检测、恢复和性能表现。**本文实现 Python TCP 原型，并与 Hash Chain、Seq+MAC 和 Ticket Only 进行对比；实验覆盖历史连续性攻击检测、攻击或断连后恢复、无效 MemoryTicket 拒绝、本地性能、并发客户端和初步代码级弱网仿真。实验结果用于验证 GMCP-R 是否能够检测历史状态异常、是否能够恢复 `last_seq` 与 `last_mem`，以及其额外开销是否可接受。

2. Related Work (相关工作)

本章从消息认证、会话恢复、哈希链、Checkpoint 恢复和状态连续性保护五个方面讨论与 GMCP-R 相关的研究基础，并进一步说明本文工作的定位。现有方法分别解决了消息完整性、连接恢复、历史篡改检测或状态防回滚中的部分问题，但在断续通信场景下，仍缺少一种同时支持历史连续性验证、恢复状态认证、低成本恢复和轻量部署的统一协议机制。

2.1 消息认证与序列保护

消息认证码广泛用于保护消息完整性与来源真实性。HMAC 将共享密钥与消息内容绑定，接收方可以验证消息是否在传输过程中被篡改。序列号机制则常用于检测旧消息重放、乱序注入和序列缺口。在轻量级安全通信中，Seq+MAC 类方案通常将序列号与消息认证码结合，使接收端能够验证单条消息的完整性以及序列单调性。

然而，这类方法主要关注单条消息及其局部顺序，通常不维护跨消息演化的历史记忆状态。也就是说，接收端可以判断某条消息的 HMAC 是否正确、序列号是否合理，但难以验证该消息是否真正接续在此前可信历史状态之后。当攻击者围绕 `prev_mem`、恢复点或历史状态构造伪造输入时，仅依赖 Seq+MAC

的方案缺少可比较的链式历史状态。因此，Seq+MAC 可以作为轻量消息认证基线，但不能直接解决本文关注的可验证历史状态恢复问题。

2.2 TLS/QUIC 会话恢复

TLS 1.3 和 QUIC 等现代安全传输协议已经提供成熟的连接建立、密钥协商和会话恢复机制。通过 PSK、Session Ticket、0-RTT 等机制，通信双方可以在断连后减少握手开销，快速恢复加密上下文或传输连接状态。这些机制对于降低重连延迟、提升网络通信效率具有重要作用。

但是，TLS/QUIC 会话恢复主要面向安全信道恢复和密钥上下文恢复，并不直接验证应用层消息历史是否连续。换言之，底层安全信道重新建立后，应用层仍可能需要确认恢复后的消息序列是否接续在此前已接受的合法历史之后。对于远程控制、工业数据采集、审计日志同步等场景，通信双方不仅需要恢复连接，还需要恢复和验证由消息序列、状态摘要和历史记忆共同刻画的应用层状态。因此，GMCP-R 并不试图替代 TLS/QUIC，而是可以作为其上层的历史状态恢复与验证机制。

2.3 哈希链与可篡改检测日志

哈希链被广泛应用于一次性口令、认证序列、安全日志和可篡改检测审计系统中。其基本思想是将当前数据与前序状态共同输入哈希函数，使每个新状态都依赖此前历史。一旦历史中的某条记录被修改，后续链式状态也会随之改变，从而暴露篡改行为。在通信场景中，哈希链能够将消息序列绑定为连续历史，为检测历史篡改、消息插入或消息删除提供基础。

不过，纯 Hash Chain 在断续通信恢复中仍存在两个限制。第一，如果恢复时需要从会话初始状态重新计算整个历史链，长会话场景下恢复成本会随历史长度线性增长。第二，哈希链本身主要提供历史绑定能力，并不天然提供恢复票据、票据时效性、nonce 一次性约束或跨会话绑定。因此，单独使用 Hash Chain 可以验证历史连续性，但难以形成完整的安全恢复流程。GMCP-R 继承哈希链的历史绑定能力，并进一步通过 MemoryTicket 与 Checkpoint 将历史状态验证扩展为可恢复、可认证和有界重放的恢复机制。

2.4 Checkpoint 与断点恢复

Checkpoint 技术在分布式系统、容错计算和状态恢复中常用于保存中间状态。当系统发生故障后，恢复过程可以从最近的 Checkpoint 继续，而不必从初始状态重新执行全部历史过程。将 Checkpoint 思想引入通信历史验证后，协议可以在恢复时从最近可信快照开始重建或验证后续状态，从而降低长会话恢复开销。

然而，传统 Checkpoint 机制主要关注系统容错和执行状态恢复，未必考虑攻击者对恢复状态的重放、回滚、篡改或跨会话迁移。在安全通信恢复中，Checkpoint 不能只是普通快照，还需要与会话上下文、序列号、记忆状态和认证标签绑定。GMCP-R 中的 Checkpoint 不仅记录 ckpt_seq 和 ckpt_mem，还与 MemoryTicket 中的 last_seq、last_mem、expire_time、nonce 和 session 信息共同接受认证与

一致性检查。因此，Checkpoint 在 GMCP-R 中不仅用于降低恢复成本，也作为验证恢复状态合法性的恢复锚点。

2.5 状态连续性与回滚保护

状态连续性和回滚保护问题在可信计算、安全存储和可信执行环境中已有较多研究。传统方案通常依赖 TPM 单调计数器、可信时间源、NVRAM、TEE 或受保护持久化存储，以防止攻击者将系统安全状态回退到旧版本。这类机制能够提供较强的硬件级状态连续性保证，适合可信平台、安全存储和高安全需求系统。

但是，在断续通信、边缘 IoT、远程设备和轻量级应用层协议中，依赖专用硬件或受保护持久化存储并不总是现实。一方面，不同设备平台的硬件能力差异较大；另一方面，跨平台部署和网络协议集成也会增加系统复杂度。因此，本文不将 GMCP-R 定位为硬件级防回滚机制的替代方案，而是在通信协议层提供一种轻量级的恢复状态连续性检查。具体而言，GMCP-R 通过 MemoryTicket 将 last_seq、last_mem、checkpoint、nonce、expire_time 和 session 信息绑定到服务器认证标签中，使服务端能够在本文威胁模型下检测恢复阶段的票据重放、历史回滚和跨会话恢复攻击。

2.6 GMCP-R 的研究定位

综上，现有研究为 GMCP-R 提供了不同方面的基础。Seq+MAC 类方法适合单包认证和序列保护，但缺少跨消息历史状态；TLS/QUIC 会话恢复可以降低连接重建成本，但不直接验证应用层历史状态；Hash Chain 能够提供历史绑定能力，但单独使用时恢复开销较高，且缺少安全恢复票据；Checkpoint 能够降低恢复成本，但需要与认证机制结合才能抵抗恶意恢复；硬件级状态连续性方案能够提供强防回滚能力，但在轻量级断续通信场景中部署成本较高。

与上述工作不同，GMCP-R 关注的是断续通信中的可验证历史状态恢复问题。其核心目标不是单纯重新建立连接，也不是仅验证单条消息，而是在攻击或断连后验证恢复点、恢复状态和最近可信快照是否仍对应同一条合法消息历史。为此，GMCP-R 将 memory chain、MemoryTicket 和 Checkpoint 结合起来：memory chain 负责历史连续性验证，MemoryTicket 负责恢复状态认证与恢复请求约束，Checkpoint 负责限制恢复时的历史重放范围。通过这种组合，GMCP-R 在不依赖专用可信硬件和完整历史日志存储的前提下，为断续通信提供一种轻量级、可验证的安全状态恢复机制。

3.System Model and Problem Definition (系模型与定)

本章给出 GMCP-R 的系统模型、通信状态、恢复状态定义、攻击模型和安全目标。本文关注的是断续通信场景下的可验证历史状态恢复问题，即通信双方在断连、攻击检测或状态失步后，不仅需要恢复通信过程，还需要验证恢复后的消息序列是否仍然接续在同一条可信历史状态链之后。

3.1 系统实体

系统包含三个基本实体：客户端 C、服务器 S 和网络攻击者 A。客户端 C 与服务器 S 通过底层 TCP 连接传输 DATA 报文。GMCP-R 不直接替代 TCP、TLS 或 QUIC 等底层连接机制，而是在应用层维护与消息历史相关的记忆状态，并在断连或攻击检测后执行历史状态恢复与验证。

客户端 C 负责构造 DATA 报文，维护本地发送序列号、前序记忆状态以及最近一次收到的 MemoryTicket。当通信中断、检测到序列号异常或收到攻击响应后，客户端可携带 MemoryTicket 发起恢复请求。

服务器 S 负责验证 DATA 报文中的 HMAC、序列号、payload_hash 和 prev_mem，并在消息被接受后更新 last_seq 与 last_mem。服务器还周期性生成 Checkpoint，并签发或验证 MemoryTicket，用于支持后续恢复过程。

网络攻击者 A 位于通信链路中，可以拦截、删除、修改、重放或注入网络消息，也可以尝试构造异常恢复请求或无效 MemoryTicket。攻击者的具体能力在 3.4 节中定义。

3.2 通信状态与记忆状态

在 GMCP-R 中，通信双方不仅维护普通会话上下文，还维护随消息序列持续演化的记忆状态。主要状态变量如下。

表 2. GMCP-R 的主要状态变量

状态变量	含义
session_id	会话标识，用于区分不同通信会话
client_id / sender_id	客户端或发送方标识，用于绑定消息来源
epoch	通信时代号，用于区分不同恢复阶段或密钥阶段
seq	单调递增的 DATA 报文序号
payload_hash	对应用层 payload 计算得到的哈希值
prev_mem	发送该消息前的前序记忆状态
last_seq	接收端已经接受的最新 DATA 报文序号
last_mem	截至 last_seq 处理后形成的当前历史记忆状态
Checkpoint	最近可信快照，包含 ckpt_seq 与 ckpt_mem
MemoryTicket	由服务器认证的恢复票据，用于绑定恢复点、恢复状态和恢复锚点

其中，seq 和 payload_hash 用于绑定单条消息内容与顺序；prev_mem 和 last_mem 用于验证当前消息是否接续在此前可信历史状态之后；Checkpoint 和 MemoryTicket 则用于攻击或断连后的恢复过程。与只验证单条消息的 Seq+MAC 不同，GMCP-R 将每条消息都纳入持续演化的记忆链中，使通信历史可以通过 last_mem 被压缩表示。

3.3 恢复点、恢复状态与恢复锚点

为了描述断续通信中的安全恢复过程，本文定义三个核心概念：恢复点、恢复状态和恢复锚点。

恢复点由 last_seq 表示，表示通信双方共同接受的最新 DATA 报文序号。恢复后，客户端应从 last_seq + 1 开始继续发送新的 DATA 报文。

恢复状态由 last_mem 表示，表示截至 last_seq 处理后形成的链式历史记忆状态。last_mem 不是普通连接状态，而是对此前消息历史、会话上下文、序列号和载荷哈希的压缩承诺。

恢复锚点由 Checkpoint 表示，包含最近可信快照 {ckpt_seq, ckpt_mem}。Checkpoint 用于缩短恢复时需要验证或重放的历史窗口，避免长会话中从初始状态重新计算整条 memory chain。

因此，一次安全恢复不是简单重新建立连接，而是要求 last_seq、last_mem 与 Checkpoint 在 MemoryTicket 认证和状态单调性约束下能够对应同一条合法消息历史。若恢复请求中的 last_seq、last_mem 或 Checkpoint 来自旧票据、跨会话票据、回滚票据或被篡改状态，则服务器应拒绝该恢复请求，并避免更新本地通信状态。

3.4 攻击模型

本文采用网络攻击者模型。攻击者 A 可以控制通信链路上的报文传输过程，包括拦截、删除、修改、重放、延迟或注入消息。具体而言，攻击者可以尝试以下攻击。

第一，消息丢弃攻击。攻击者删除某些 DATA 报文，使接收端观察到序列号缺口。

第二，消息篡改攻击。攻击者修改 payload、payload_hash、seq、prev_mem 或其他关键字段，试图使接收端接受被篡改消息。

第三，消息重放攻击。攻击者重新发送旧 DATA 报文，试图使旧消息被再次接受为当前有效消息。

第四，历史状态伪造攻击。攻击者伪造 prev_mem 或诱导通信双方接受错误的 last_mem，使后续通信建立在不一致历史状态之上。

第五，恢复票据攻击。攻击者尝试使用过期、重复、篡改、回滚或跨会话的 MemoryTicket 发起恢复请求，从而诱导服务器恢复到错误历史状态。

本文假设攻击者不能破坏 SHA-256 和 HMAC-SHA256 等密码学原语，不能获得通信双方的共享密钥或服务器票据认证密钥，也不能完全控制客户端或服务端。本文的安全结论均限定在上述攻击模型和已测试攻击场景之内。

3.5 安全目标

GMCP-R 的目标是在断续通信中提供可验证的历史状态恢复能力。具体安全目标如下。

表 3. GMCP-R 的主要安全目标

安全目标	含义
消息完整性	任何对 payload 或关键元数据的篡改都应触发认证失败
历史连续性	消息历史应形成可验证的链式状态，prev_mem 伪造或历史断裂应被检测
重放抵抗	旧 DATA 报文或旧 MemoryTicket 不能被再次接受为当前有效状态
回滚抵抗	恢复过程不能使通信状态退回到低于当前要求的 last_seq 或旧 last_mem
票据真实性证	MemoryTicket 必须由服务器认证生成，字段被改动后应无法通过验

安全目标	含义
恢复状态一致性	恢复后的 last_seq、last_mem 与 Checkpoint 应对应同一条合法消息历史
可接受开销	协议应在提供历史状态恢复能力的同时保持可接受的计算、通信和状态存储开销

上述目标中，消息完整性和重放抵抗是基础安全目标；历史连续性、回滚抵抗和恢复状态一致性是 GMCP-R 区别于普通 Seq+MAC 或 Ticket Only 机制的核心目标；可接受开销则用于约束协议在断续通信、边缘设备和 IoT 场景中的部署可行性。

4. GMCP-R Protocol Design（协议设计）

4.1 协议概览

GMCP-R 是一种面向断续通信的记忆链安全恢复协议，其目标是在通信中断、攻击检测或状态失步后，帮助通信双方恢复并验证由 last_seq、last_mem 与 Checkpoint 共同刻画的历史记忆状态。GMCP-R 不直接替代底层 TCP、TLS 或 QUIC 的连接恢复机制，而是在应用层维护可验证的历史状态，使恢复后的消息能够继续接续在可信历史之后。

从协议流程看，GMCP-R 包含四个阶段。第一，在正常 DATA 通信阶段，客户端为每条消息计算 payload_hash，携带 seq 和 prev_mem，并使用 HMAC 保护关键字段；服务器验证 HMAC、序列号和 prev_mem 后，更新 last_seq 与 last_mem。第二，在 Checkpoint 生成阶段，服务器周期性记录最近可信快照 {ckpt_seq, ckpt_mem}，用于限制后续恢复时的历史重放范围。第三，在 MemoryTicket 签发阶段，服务器将 last_seq、last_mem、Checkpoint、过期时间、nonce 和会话上下文绑定到认证票据中，形成可用于恢复的状态承诺。第四，在安全恢复阶段，当通信中断或攻击被检测后，客户端携带最近的 MemoryTicket 发起恢复请求；服务器验证票据认证标签、时效性、nonce、会话绑定和状态单调性后，返回可恢复状态或明确拒绝原因。

因此，GMCP-R 的核心思想不是仅恢复一条网络连接，而是恢复并验证后续通信所需的历史状态起点。DATA 阶段持续推进 last_seq 与 last_mem，Checkpoint 提供最近可信恢复锚点，MemoryTicket 则将恢

复点、恢复状态和恢复锚点绑定为服务器认证的恢复凭证。只有当这些字段能够对应同一条合法消息历史时，通信双方才继续从 last_seq + 1 推进新的 DATA 报文。

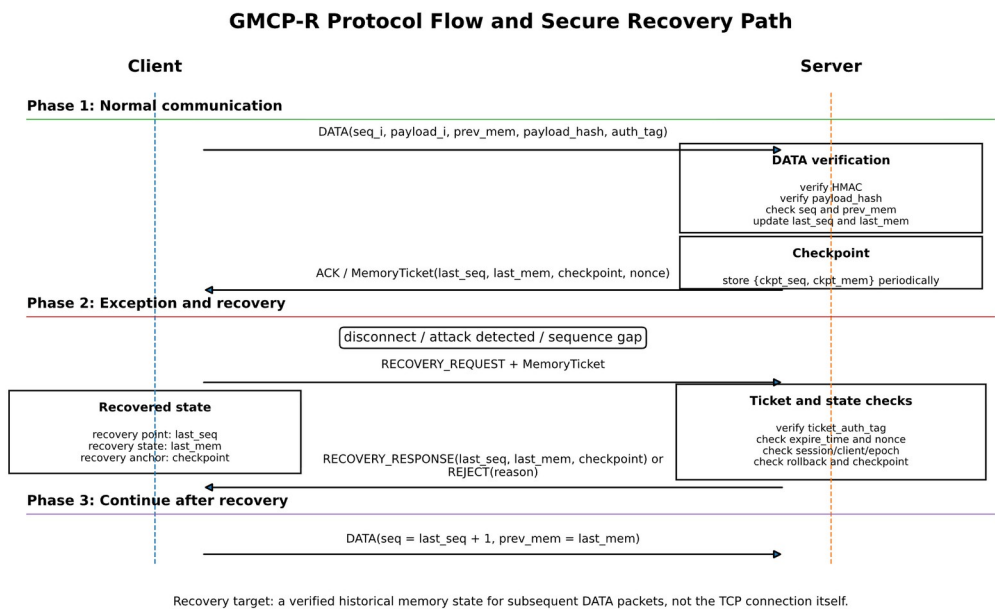


图 1. GMCP-R 协议整体流程与安全恢复路径

4.2 Memory Chain Construction（记忆链构造）

GMCP-R 使用随消息逐步演化的 memory chain 表示通信历史状态。记 M_0 为会话建立时生成或确认的初始记忆状态， M_i 表示第 i 条 DATA 报文被接收端接受后形成的记忆状态， h_i 表示该报文载荷的哈希值。其计算过程定义为：

$$h_i = H(\text{payload}_i)$$

$$M_i = H(\text{Enc}(M_{i-1}, \text{sid}, e, \text{seq}_i, h_i, \text{sender}))$$

其中，H 表示密码学哈希函数；Enc(·) 表示对各字段进行长度明确、顺序固定的无歧义编码；sid 表示会话标识；e 表示当前 epoch；seq_i 表示第 i 条 DATA 报文的单调递增序列号；sender 表示消息发送方标识。第 i 条报文中的 prev_mem 字段应满足：

$$\text{prev_mem}_i = M_{i-1}$$

接收端收到第 i 条 DATA 报文后，首先验证报文认证标签和 payload_hash，然后检查报文序号是否满足预期的单调递增关系，并比较报文中的 prev_mem 与本地保存的 last_mem。只有在上述验证均通过时，接收端才计算新的记忆状态 M_i，并执行以下状态更新：

$$\text{last_seq} \leftarrow \text{seq}_i$$

$$\text{last_mem} \leftarrow M_i$$

该设计将前序记忆状态、会话上下文、消息序号、载荷摘要和发送方身份共同绑定到新的记忆状态中。与仅对 payload 进行链式哈希的简单方案相比，GMCP-R 的 memory chain 能够区分不同会话、不同 epoch 和不同消息位置，降低跨会话混淆及乱序注入的风险。

在哈希函数满足抗碰撞性的假设下，如果攻击者修改任一被绑定字段，则后续记忆状态的输入也会发生变化。攻击者若希望不同消息历史产生相同的可接受记忆状态，需要构造哈希碰撞；若希望被修改的报文仍通过认证，还需要伪造有效的 HMAC 标签。因此，在哈希函数抗碰撞性和 HMAC 不可伪造性的假设下，攻击者难以在不触发验证失败的情况下修改已绑定的通信历史。

需要说明的是，memory chain 主要提供历史状态绑定与连续性验证。旧消息重放由 seq 检查识别，恢复阶段的历史回滚则进一步由 MemoryTicket、nonce 和状态单调性约束进行检测。

图 2 展示了 GMCP-R 中 memory chain 的状态演化过程。每条 DATA 报文携带发送前的 prev_mem，接收端将其与本地 last_mem 比较。验证通过后，接收端根据当前报文字段计算新的 M_i，并更新 last_seq 与 last_mem，使后续消息必须接续在该状态之后。

GMCP-R Memory Chain State Evolution

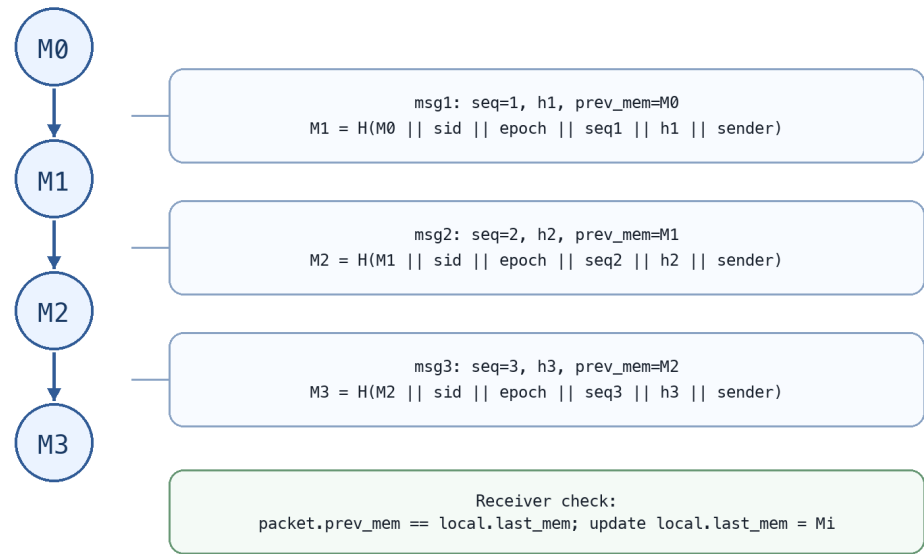


图 2. GMCP-R 的记忆链状态演化过程

4.3 DATA 报文格式

GMCP-R 使用 DATA 报文传输应用层消息，并在每条报文中携带会话上下文、序列状态、载荷摘要和前序记忆状态。接收端只有在报文结构、消息认证、序列单调性和历史连续性均通过验证后，才接受该报文并更新本地状态。

图 3 给出了 DATA 报文中字段分组与验证关系。该图强调 prev_mem 属于链式状态输入，而 auth_tag 需要覆盖会话上下文、顺序状态、内容绑定和链式状态等关键字段。

GMCP-R DATA Packet Format

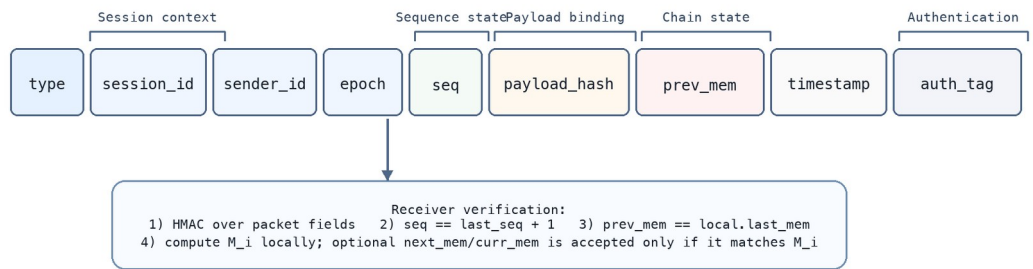


图 3. GMCP-R DATA 报文格式与验证字段。

表 4. GMCP-R DATA 报文字段

字段类别	字段	是否必需	作用
报文类型	type	是	标识当前报文为 DATA 类型，防止不同类型报文被错误处理。
会话上下文	session_id	是	将报文绑定到特定通信会话，防止跨会话混淆。
发送方标识	sender_id	是	标识消息发送方，并将发送方身份绑定到记忆链和认证计算中。
状态阶段	epoch	是	区分不同恢复阶段或密钥阶段，防止旧阶段报文被当前阶段接受。
顺序状态	seq	是	表示 DATA 报文的单调递增序号，用于检测重放、旧包和序列缺口。
应用载荷	payload	是	承载应用层数据。
载荷摘要	payload_hash	是	对 payload 计算得到的哈希值，用于验证载荷完整性并参与记忆链更新。
链式状态	prev_mem	是	表示发送该消息前的前序记忆状态，用于与接收端本地 last_mem 比较。

字段类别	字段	是否必需	作用
时间信息	timestamp	可选	用于审计排序或辅助实现时间策略；若未实现时间验证，则不作为核心接受条件。
认证字段	auth_tag	是	对会话上下文、顺序状态、载荷摘要和前序记忆状态进行 HMAC 认证。

其中，第 i 条 DATA 报文中的 `prev_mem` 应等于发送该报文前的记忆状态 M_{i-1} 。接收端收到报文后，需要验证：

$$P.\text{prev_mem} = \text{local.last_mem}$$

验证通过后，接收端根据 `session_id`、`epoch`、`seq`、`payload_hash`、`sender_id` 和本地 `last_mem` 计算新的记忆状态 M_i 。

第 i 条 DATA 报文的认证标签定义为：

$$\text{auth_tag}_i = \text{HMAC_K}(\text{Enc}(\text{type}, \text{session_id}, \text{sender_id}, \text{epoch}, \text{seq}_i, \text{payload_hash}_i, \text{prev_mem}_i))$$

其中， K 表示客户端与服务器共享的消息认证密钥。若报文显式携带 `timestamp`，则 `timestamp` 也应纳入 `Enc()` 和 HMAC 覆盖范围。

当前原型不要求 DATA 报文显式携带 `next_mem` 或 `curr_mem`。接收端可以根据已经通过验证的报文字段本地计算 M_i ，因此不传输该字段可以减少冗余并避免发送方提供的结果被直接信任。在调试、审计或跨节点状态同步场景中，可以将 `next_mem` 作为扩展字段；若显式携带，接收端仍必须独立计算 M_i ，并在两者不一致时拒绝该报文。

Algorithm 1. GMCP-R DATA Packet Verification

Input: packet P , <code>expected_session_id</code> , <code>last_seq</code> , <code>last_mem</code> , key K
Output: accept/reject decision and updated memory state
1: if required fields are missing then reject
2: if $P.\text{type} \neq \text{DATA}$ then reject
3: if $P.\text{session_id} \neq \text{expected_session_id}$ then reject
4: $h' = H(P.\text{payload})$
5: if $h' \neq P.\text{payload_hash}$ then reject
6: $\text{tag}' = \text{HMAC}(K, P \text{ without auth_tag})$

Input: packet P, expected_session_id, last_seq, last_mem, key K
7: if tag' != P.auth_tag then reject
8: if P.seq <= last_seq then reject as replay or stale packet
9: if P.seq > last_seq + 1 then reject as sequence gap
10: if P.prev_mem != last_mem then reject as history discontinuity
11: M_i = H(last_mem P.session_id P.epoch P.seq P.payload_hash P.sender_id)
12: if P.next_mem is present and P.next_mem != M_i then reject as memory-result mismatch
13: update last_seq = P.seq and last_mem = M_i
14: accept P

4.4 Checkpoint 机制

Checkpoint 是周期性状态快照，记录服务器已经验证并接受的近端历史状态。默认配置下，每处理 100 条 DATA 报文创建一个 Checkpoint；快照字段包括 session_id、epoch、ckpt_seq、ckpt_mem、created_at 和 checkpoint_auth_tag。恢复时，服务器可以从最近可信 Checkpoint 开始验证或重放后续少量消息，而不必从会话初始状态重建全部历史。因此，Checkpoint 将纯 Hash Chain 的恢复成本从 O(n) 限制为 O(k)，其中 n 为会话历史长度，k 为 checkpoint 间隔或最近快照之后的待验证后缀长度。

表 5. Checkpoint 核心字段与作用

字段	作用
session_id / epoch	将快照绑定到具体会话和状态阶段，避免跨会话或跨 epoch 恢复。
ckpt_seq	记录快照覆盖到的最新消息序号，作为恢复锚点的序列位置。
ckpt_mem	记录 ckpt_seq 对应的记忆链状态，作为后续恢复验证起点。
created_at	辅助判断快照新旧和审计恢复过程。
checkpoint_auth_tag	服务器对快照字段计算认证标签，防止快照被篡改。

在恢复算法中，Checkpoint 不单独决定恢复成功，而是与 MemoryTicket 中的 last_seq 和 last_mem 联合使用。若 ckpt_seq > last_seq，或 ckpt_mem 无法作为 last_mem 所属历史链的可信前缀，服务端应拒绝恢复请求。

4.5 MemoryTicket 机制

表 6. MemoryTicket 核心字段与安全作用

字段	作用
session_id / client_id / epoch	绑定恢复请求的会话、客户端和时代上下文
last_seq / last_mem	记录票据签发时的最新序列号与记忆状态
ckpt_seq / ckpt_mem	记录最近可信快照，支持有界恢复
expire_time	限制票据有效期，防止长期滥用
ticket_nonce	一次性随机数，用于检测票据重放
key_version	支持后续密钥轮换或版本管理
server_auth_tag	服务器 HMAC 签名，保护票据字段完整性

MemoryTicket 由服务器在正常通信或安全恢复后签发，用于把恢复点、恢复状态和恢复锚点绑定为一个短期可验证凭证。票据不是简单的会话恢复 token，而是包含 last_seq、last_mem、ckpt_seq、ckpt_mem、expire_time、ticket_nonce 与 key_version 的状态承诺。

$$\text{tag_T} = \text{HMAC}_{\{K_s\}}(\text{Enc}(\text{sid}, \text{cid}, \text{epoch}, \text{last_seq}, \text{last_mem}, \text{ckpt_seq}, \text{ckpt_mem}, \text{exp}, \text{nonce}, \text{key_version}))$$

其中，K_s 为服务器侧票据认证密钥。expire_time 限制票据有效期，ticket_nonce 用于一次性消费检测，key_version 支持服务器侧密钥轮换。服务端验证恢复请求时必须重新计算 tag_T，并同时检查 nonce 是否已使用、会话上下文是否匹配、last_seq 是否满足最小可接受恢复序号，以及 checkpoint 是否与恢复状态结构一致。

其中，K_s 为服务器侧票据认证密钥，cid 表示客户端标识，exp 表示过期时间，nonce 表示一次性票据随机数，kv 表示密钥版本。由于 tag_T 覆盖会话、客户端、时代号、恢复序列、记忆状态、checkpoint、过期时间、nonce 和密钥版本，攻击者修改任一字段都会导致服务器重新计算的认证标签不匹配，从而使恢复请求被拒绝。

4.6 恢复流程

1. 客户端检测到断连、序列号间隙或攻击响应后，准备最近一次收到的 MemoryTicket。
2. 客户端发送 RECOVERY_REQUEST，并附带 ticket、session_id、client_id、epoch 等上下文。
3. 服务器验证票据字段、HMAC 签名、过期时间、nonce 是否已使用、session_id/client_id/epoch 是否匹配，以及 last_seq 是否满足单调性约束。
4. 验证通过后，服务器返回恢复状态 (last_seq、last_mem、checkpoint_seq、checkpoint_mem)；验证失败时返回明确拒绝原因，并拒绝更新状态。

5. 客户端从 last_seq + 1 继续通信，并用恢复后的 last_mem 作为后续 memory chain 的起点。

图 4 展示了基于 MemoryTicket 的安全恢复流程。该流程的关键在于服务器先验证票据真实性和状态单调性，再返回恢复后的 last_seq 与 last_mem；若任一检查失败，恢复请求被显式拒绝。

MemoryTicket-Based Secure Recovery Procedure

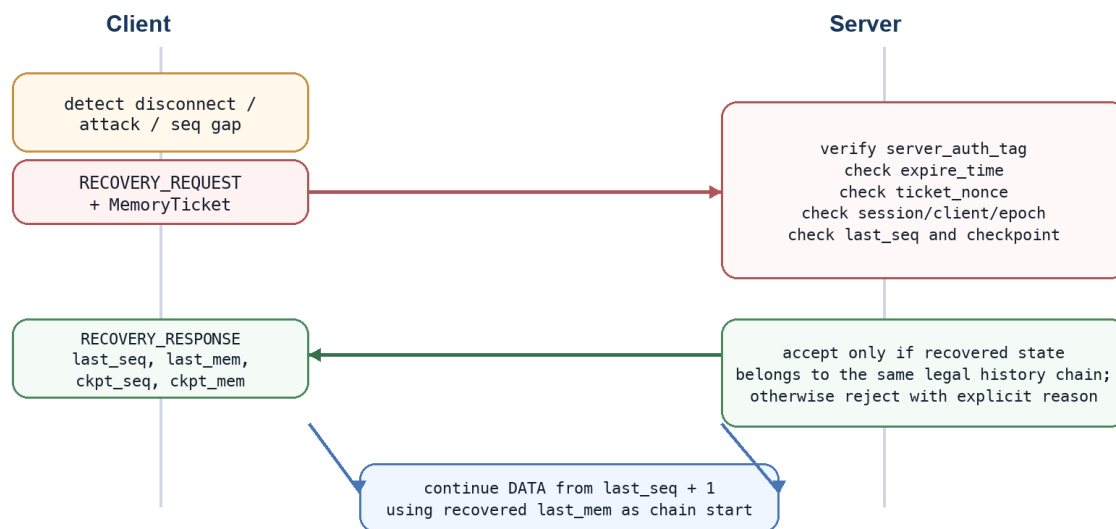


图 4. 基于 MemoryTicket 的安全恢复流程。

因此，GMCP-R 的恢复对象不是应用层 payload 全文，也不是单纯网络连接句柄，而是能够作为后续安全通信起点的历史记录状态：last_seq 给出恢复点，last_mem 给出该恢复点对应的链式状态，Checkpoint 给出可验证的近端快照。三者一致时，通信双方才能继续推进新的 DATA 报文。

Algorithm 2. MemoryTicket-Based Recovery

Input: recovery request R, MemoryTicket T, server state S
Output: recovery response or explicit rejection reason
1: check required ticket fields are present; otherwise reject with "missing ticket field"
2: verify T.server_auth_tag using server key; otherwise reject with "invalid ticket auth tag"
3: if T.expire_time < current_time then reject with "ticket expired"
4: if T.ticket_nonce has been consumed then reject with "ticket replay detected"
5: if T.session_id, T.client_id, or T.epoch mismatches R then reject with "session_id/client_id/epoch mismatch"
6: if T.last_seq < S.min_last_seq then reject with "rollback detected"
7: if T.checkpoint_seq > T.last_seq then reject with "checkpoint_seq exceeds last_seq"
8: consume T.ticket_nonce
9: return recovery state (last_seq, last_mem, checkpoint_seq, checkpoint_mem)

4.7 Secure Recovery Semantics（安全恢复语义）

GMCP-R 中“恢复”的语义不是恢复 TCP 连接句柄，也不是恢复完整 payload 历史，而是恢复可作为后续安全通信起点的历史记忆状态。last_seq 决定通信双方从哪个消息序号之后继续，last_mem 决定该恢复点对应的链式历史状态，Checkpoint 则限制恢复验证或历史重放的范围。

一次恢复被接受，需要满足三个条件：第一，MemoryTicket 能通过服务器认证标签验证，说明恢复状态字段未被篡改；第二，last_seq、last_mem、ckpt_seq 和 ckpt_mem 在结构上相互一致，说明恢复状态可解释为同一条合法历史链上的状态；第三，nonce、expire_time 和 min_last_seq 等检查通过，说明该恢复请求不是旧票据重放或历史回滚。只有三者同时成立，客户端才允许从 last_seq + 1 继续发送 DATA 报文。

5. Security Analysis（安全分析）

5.1 消息完整性

Claim. Message Integrity. 在 HMAC-SHA256 满足不可伪造性的假设下，攻击者难以修改 DATA 报文中的 payload、seq、prev_mem 或会话上下文字段而使接收端接受该报文。

Proof Sketch. 每个 DATA 报文的 auth_tag 覆盖除认证标签自身外的关键字段，包括 payload_hash、seq、prev_mem、session_id、epoch 与 sender_id。攻击者若修改任一字段，则服务器重新计算得到的 HMAC 输入发生变化。若修改后仍能通过验证，则意味着攻击者在未知共享密钥 K 的情况下伪造了有效 HMAC 标签，与 HMAC 不可伪造性假设矛盾。

5.2 历史连续性

Property 1. History Continuity. 在哈希函数 H 满足抗碰撞性，且 HMAC 不可伪造的假设下，攻击者难以构造一条与已接受历史不同的消息序列，却使接收端得到相同且可接受的 last_mem。

Proof Sketch. 设两条历史序列在某一位置 j 首次不同，但最终均被接收端接受并得到相同记忆状态 M_i。由于 M_i 递归依赖 M_{i-1}、session_id、epoch、seq_i、payload_hash_i 和 sender_id，位置 j 的差异会改变后续所有记忆状态输入。若最终状态仍相同，则要么存在一次哈希碰撞，要么攻击者伪造了覆盖差异字段的

HMAC 标签。根据 H 的抗碰撞性和 HMAC 的不可伪造性，该事件概率可忽略。因此，不同历史难以产生相同可接受记忆状态。

5.3 Ticket Unforgeability（票据不可伪造）

Property 2. Ticket Unforgeability. 在 HMAC 安全假设下，攻击者不能修改 MemoryTicket 中的 last_seq、last_mem、checkpoint、expire_time、nonce 或上下文字段而仍通过服务端验证。

Proof Sketch. MemoryTicket 的 server_auth_tag 覆盖 sid、cid、epoch、last_seq、last_mem、ckpt_seq、ckpt_mem、exp、nonce 和 key_version。攻击者修改任一字段都会改变 HMAC 输入；若修改后仍通过验证，则等价于成功伪造 HMAC 标签，与 HMAC 不可伪造性假设矛盾。

5.4 重放与回滚抵抗

Property 3. Rollback Resistance. 若服务端维护最小可接受恢复序号 min_last_seq，并记录已消费 ticket_nonce，则任何 last_seq < min_last_seq 的旧票据或重复使用票据均会被拒绝。

Proof Sketch. 服务端在恢复算法中显式检查 $T.last_seq < S.min_last_seq$ ，并在该条件成立时返回 rollback detected；同时，已消费的 ticket_nonce 会被记录并用于拒绝重复恢复请求。因此，攻击者无法利用旧票据将服务端恢复到低于 min_last_seq 的历史位置，也无法通过重复提交同一有效票据绕过状态单调性检查。报文级重放还会被 $seq \leq last_seq$ 检测，避免恢复后接受旧 DATA 报文。

5.5 Recovery State Consistency（恢复状态一致性）

Property 4. Recovery State Consistency. 若服务端恢复算法接受一个恢复请求，则返回的 last_seq、last_mem、checkpoint_seq 和 checkpoint_mem 必须属于同一会话、同一 epoch 下的一条合法历史链；服务端不会接受不属于合法历史链的恢复状态。

Proof Sketch. 服务端首先验证 MemoryTicket 的 HMAC，因此攻击者不能单独替换 last_seq、last_mem 或 checkpoint 字段。随后，恢复算法检查 session_id、client_id 与 epoch 是否匹配请求上下文，检查 $checkpoint_seq \leq last_seq$ ，并检查 last_seq 不低于服务端维护的 min_last_seq。由于 last_mem 是由 memory chain 递归计算得到的状态，且 checkpoint_mem 是该链上的可信近端快照，任何与合法历史链不一致的恢

复状态都会在票据认证、上下文绑定、checkpoint 结构检查或状态单调性检查中失败。因此，被接受的恢复状态可以作为后续 DATA 报文的安全起点。

需要强调的是，本文目前提供的是工程实现与实验场景下的安全分析，而非完整形式化证明。更严格的证明可在后续工作中借助 ProVerif、Tamarin 或类似工具完成 [REF-PROVERIF], [REF-TAMARIN]。

6. Implementation and Experimental Setup（实现与实验设置）

6.1 原型实现

项目使用 Python 3.11 实现 GMCP-R 原型，核心模块包括 memory.py、packet.py、ticket.py、checkpoint_manager.py 与 protocol.py。密码学操作采用 SHA-256 与 HMAC-SHA256。实验包含真实 TCP baseline 对比、MemoryTicket 无效票据拒绝、本地纯计算性能、并发客户端测试和代码级弱网仿真。

表 7. 实验环境与参数概览

类别	配置
客户端环境	Python 3.11, macOS / Darwin 24.6.0（部分实验元数据记录）
服务器环境	Linux CentOS-7.6.1810-x64, 4 核 CPU, 4GB 内存, VPC 网络
传输协议	TCP/IP；GMCP-R 默认端口 9000, baseline 默认端口 9001
基线协议	Hash Chain、Seq+MAC、Ticket Only
消息数量	100、500、1000；并发实验为每客户端 200 条
载荷大小	64、128、256、512、1024 bytes（按实验类型不同取子集）
攻击类型	drop、modify、replay、prev_mem；票据实验另含 expired、replayed、tampered、rollback、wrong_session
主要指标	吞吐量、RTT、攻击检测率、误接受率、误拒绝率、恢复成功率、并发成功率

数据来源：docs/experiment_methodology.md、results/**/*.csv 与脚本元数据。

6.2 Baseline 与测试矩阵

本文设置三类 baseline：Hash Chain、Seq+MAC 和 Ticket Only。Hash Chain 用于比较纯链式历史验证能力，Seq+MAC 用于比较传统消息认证与序列号保护，Ticket Only 用于比较仅依赖恢复凭证的方案。真实

baseline 对比实验覆盖 4 种协议、5 类攻击类型、3 种消息数量、2 种载荷大小与 3 次重复，共 360 条记录。

6.3 Attack and Recovery Scenarios（攻击与恢复场景）

攻击场景包括 drop、modify、replay 和 prev_mem。drop 用于模拟消息缺失与序列号间隙；modify 用于模拟 payload 或认证字段篡改；replay 用于模拟旧消息重新注入；prev_mem 用于直接检验历史连续性字段是否被篡改。恢复场景覆盖 drop recovery、modify recovery、replay recovery、prev_mem recovery 和 disconnect recovery。票据攻击另行覆盖 expired_ticket、replayed_ticket、tampered_ticket、rollback_ticket 和 wrong_session_ticket 五类无效 MemoryTicket。

6.4 Metrics（评价指标）

主要指标包括攻击检测率、误接受率、误拒绝率、恢复成功率、恢复后 memory 一致率、最终序列一致性、恢复延迟、额外恢复消息数、额外恢复字节数、吞吐量、RTT 和并发成功率。对于代码级弱网仿真，本文只将其作为恢复行为的补充观察，不将其作为真实网络损伤下的核心结论。

7. Results（实验结果）

7.1 Baseline Comparison

表 8 汇总了真实 baseline 对比实验的主要结果。GMCP-R 与 Hash Chain 在测试攻击场景中均达到 100% 攻击检测率和 0% 误接受率；Seq+MAC 因缺少历史记忆链，对 prev_mem 类攻击存在缺口；Ticket Only 仅保护会话票据，无法完整验证消息内容与历史连续性。

表 8. Baseline 协议对比结果

协议	正常吞吐量 (msg/s)	正常 RTT (ms)	攻击	接受率	拒绝
GMCP-R	9,621	0.103	100%	0%	0%
Hash Chain	11,786	0.096	100%	0%	0%
Seq+MAC	11,727	0.091	75%	25%	0%
Ticket Only	13,376	0.087	50%	50%	0%

数据来源：results/real_baseline_comparison/summary_real_baseline_comparison_v2.csv。

本节采用与表 8 一致的 v2 汇总口径报告描述性统计。可以看到，GMCP-R 与 Hash Chain 在测试攻击集合中的检测率均为 100%，而 Seq+MAC 和 Ticket Only 分别存在 25% 与 50% 的误接受率。考虑到当前真实 baseline 每个配置的重复次数仍较少，本稿暂不报告正式统计推断，后续扩展重复次数后再补充显著性检验。

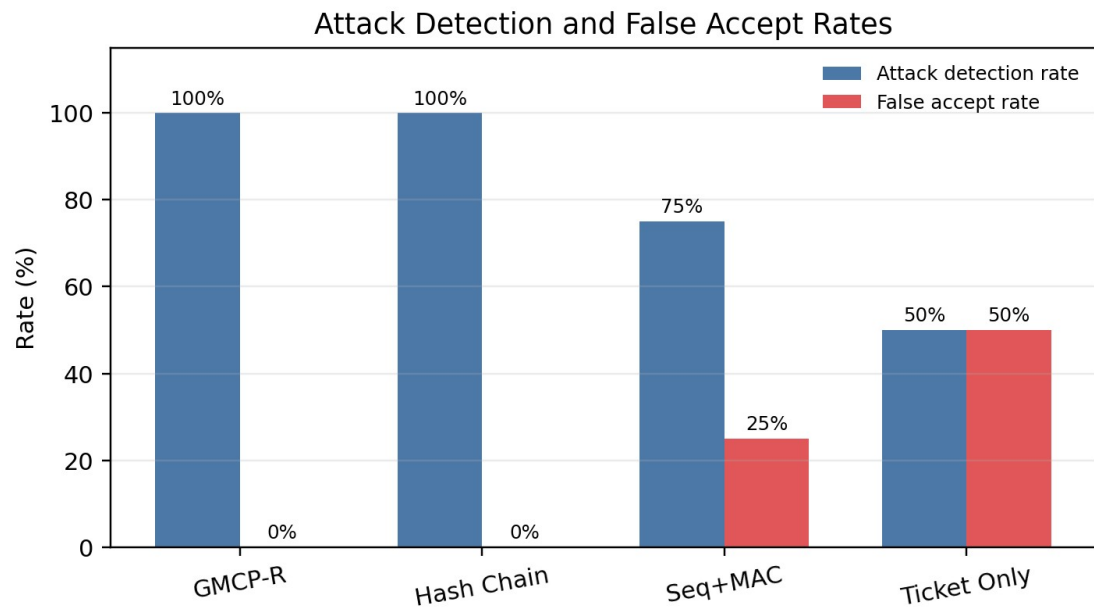


图 5. 各协议攻击检测率与误接受率对比。

数据来源：results/real_baseline_comparison/summary_real_baseline_comparison_v2.csv。

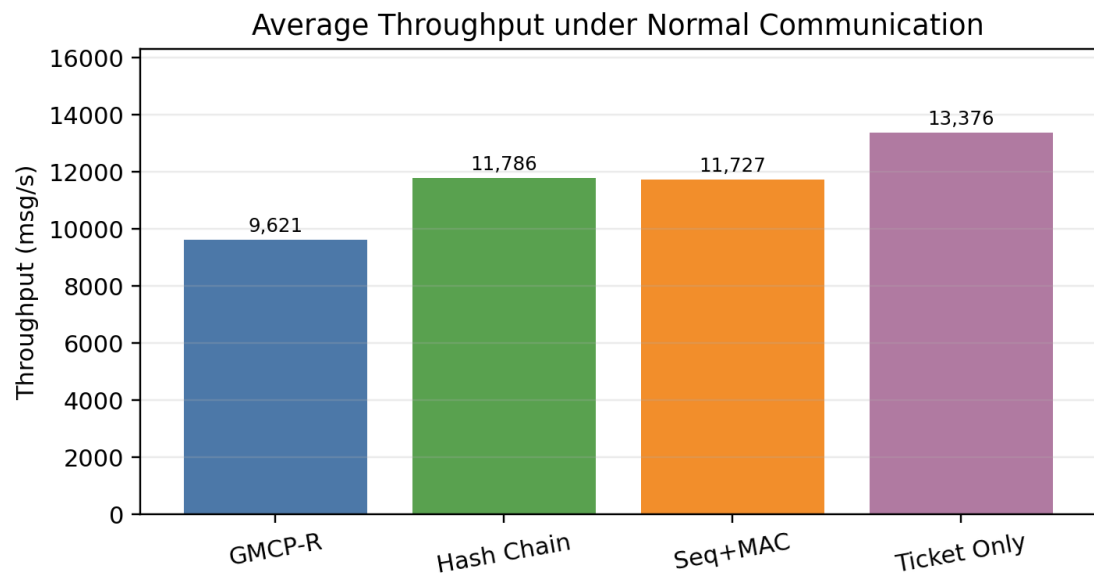


图 6. 正常通信条件下各协议平均吞吐量对比。

数据来源：results/real_baseline_comparison/summary_real_baseline_comparison_v2.csv。

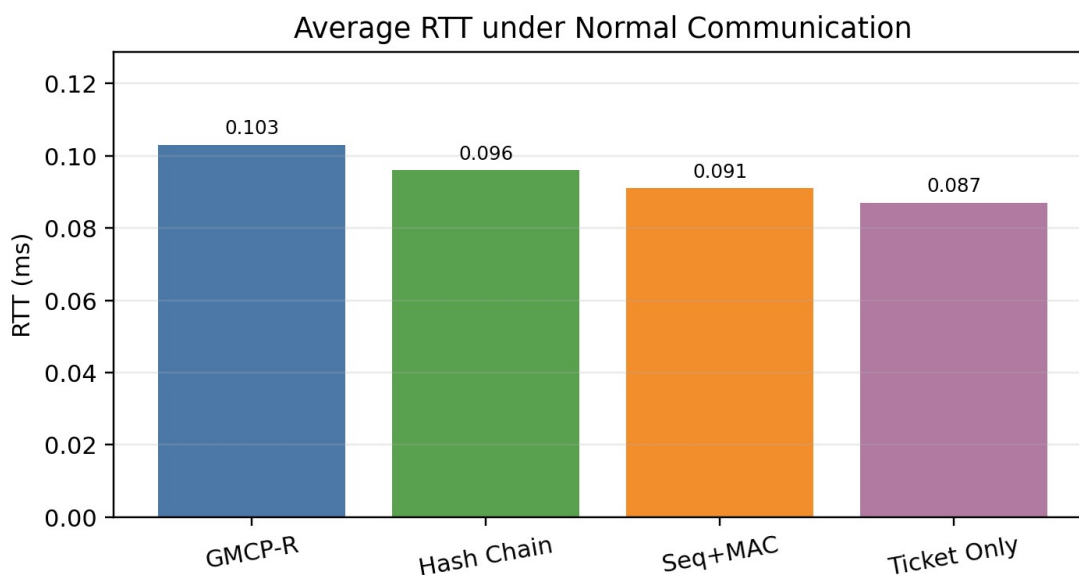


图 7. 正常通信条件下各协议平均 RTT 对比。

数据来源：results/real_baseline_comparison/summary_real_baseline_comparison_v2.csv。

上述差异主要来自各协议绑定历史状态的能力不同。Seq+MAC 能验证单包 HMAC 和序列单调性，因此可检测 payload 篡改、旧序列重放等攻击，但它不维护跨消息 memory chain；当攻击围绕 prev_mem 或历史连续性构造时，接收端缺少可比较的链式状态，因而出现误接受。Ticket Only 更偏向恢复凭证验证，能够表达“某个票据是否有效”，但不将每条 DATA 报文的载荷、序列和历史记忆状态绑定到恢复检查中，因此误接受率更高。GMCP-R 与 Hash Chain 在测试攻击集中的检测率相同，二者都依赖链式历史状态；GMCP-R 的区别不在于检测率高于 Hash Chain，而在于额外提供 MemoryTicket 与 Checkpoint 支持的安全恢复机制。

7.2 Secure Recovery after Attacks and Disconnections

本节直接回答“是不是恢复了”的问题。实验在真实 TCP 原型上构造 drop、modify、replay、prev_mem 和 disconnect 五类异常场景。客户端在检测到攻击、序列缺口或断连后发起 RECOVERY_REQUEST，服务器基于 MemoryTicket 和 Checkpoint 返回恢复状态，并要求客户端从 last_seq + 1 继续通信。

表 9. 攻击后安全恢复结果

场景	攻击检测率	恢复成功率	memory 一致率	seq 一致率	恢复延迟 (ms)	额外消息	额外字节
drop	100%	100%	100%	100%	173.8	2	1373
modify	100%	100%	100%	100%	179.4	2	1370
replay	100%	100%	100%	100%	180.3	2	1371
prev_mem	100%	100%	100%	100%	175.6	2	1391
disconnect	100%	100%	100%	100%	345.6	2	1381

数据来源: results/real_recovery/summary_real_recovery.csv。

结果显示，五类场景下 GMCP-R 的攻击检测率、恢复成功率、恢复后 memory 一致率和最终序列一致率均为 100%。恢复过程平均只额外使用 2 条消息，额外字节约 1.37–1.39 KB。disconnect 场景恢复延迟较高，主要反映连接中断后的等待和重连开销；drop、modify、replay 与 prev_mem 场景的恢复延迟集中在约 174–180 ms。

Secure Recovery after Attacks and Disconnections

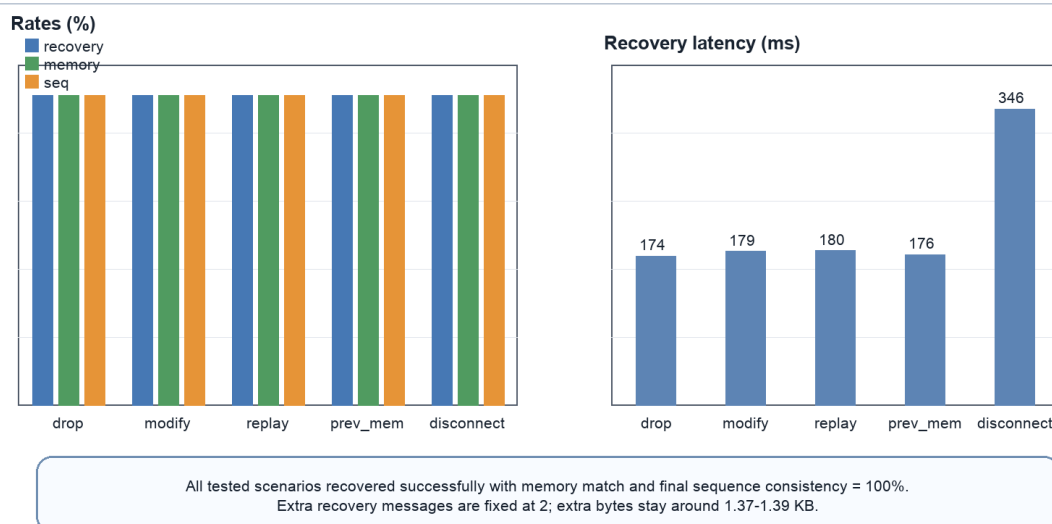


图 8. 攻击和断连后的安全恢复结果。

7.3 Invalid MemoryTicket Rejection

为验证恢复票据机制的安全性，实验构造了过期、重放、篡改、回滚和跨会话五类无效 MemoryTicket。结果表明，所有无效票据均被服务器拒绝，并返回明确拒绝原因。

表 10. MemoryTicket 无效票据拒绝结果

票据类型	恢复成功率	服务器拒绝原因	验证机制
expired_ticket	0% (全部拒绝)	ticket expired	过期时间校验
replayed_ticket	0% (全部拒绝)	ticket replay detected; rollback detected in repeated-use cases	nonce 唯一性与状态单调性 校验
tampered_ticket	0% (全部拒绝)	invalid ticket auth tag	HMAC 签名校验
rollback_ticket	0% (全部拒绝)	checkpoint_seq exceeds last_seq	结构一致性与防回滚校验
wrong_session_ticket	0% (全部拒绝)	session_id mismatch	会话标识绑定校验

数据来源：results/real_ticket_recovery/real_ticket_recovery_results.csv。

五类无效票据分别对应恢复机制中的关键安全检查：expired_ticket 检查票据时效性，replayed_ticket 检查 nonce 一次性与状态单调性，tampered_ticket 检查服务器认证标签，rollback_ticket 检查 checkpoint 与 last_seq 的结构一致性，wrong_session_ticket 检查会话绑定。实验结果说明，在这些测试场景下，MemoryTicket 不只是恢复凭证，也承担恢复状态完整性、重放检测和防回滚检查的入口。

7.4 Recovery Cost and Checkpoint Effectiveness

本节用于说明 Checkpoint 的价值。由于真实网络恢复延迟受公网抖动影响较大，本文同时报告协议级恢复成本模型，用于比较不同协议在恢复时需要重放的历史长度和额外字节数。该结果用于解释机制开销，不替代真实 TCP 端到端性能结论。

表 11. 恢复成本与 Checkpoint 效果对比

协议	恢复延迟(ms)	memory 恢复率	安全快速恢复率	额外字节	重放消息数
GMCP-R	3.35	100%	100%	6,592	63.3
Hash Chain	30.98	100%	0%	369,600	1155.0
Seq+MAC	1.60	0%	0%	128	0.0
Ticket Only	3.10	0%	0%	256	0.0

数据来源：results/baseline/summary_baseline_comparison.csv（protocol-level simulation）。

可以看到，GMCP-R 在保持 100% memory 恢复率和 100% 安全快速恢复率的同时，将平均重放消息数限制在约 63 条，额外恢复字节约 6.6 KB；纯 Hash Chain 需要重放约 1,155 条消息，额外字节约 369.6 KB。该差异体现了 Checkpoint 将恢复成本限制在近端后缀而非完整历史上的作用。

Recovery Cost and Checkpoint Effectiveness

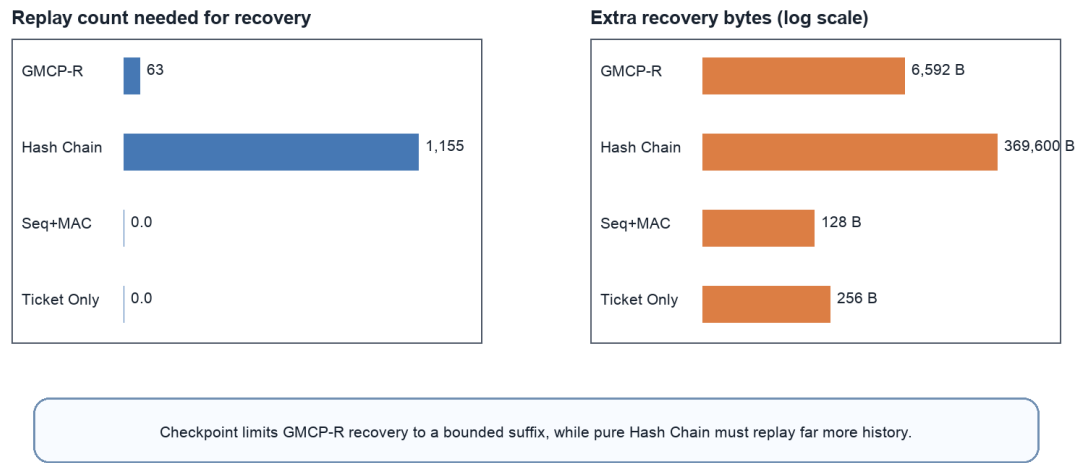


图 9. Checkpoint 对恢复重放成本和额外字节的影响。

7.5 Performance Benchmark

本地纯计算性能基准测试排除了网络往返影响，主要衡量协议构造、认证和验证逻辑的开销。GMCP-R 的平均吞吐量约为 65,931 msg/s，平均端到端处理延迟约为 14.96 μ s。与更轻量的 Ticket Only、Hash Chain 和 Seq+MAC 相比，GMCP-R 的开销主要来自 payload_hash、memory chain 更新、字段覆盖更完整的 HMAC 以及 Checkpoint 管理。

表 12. 性能基准测试结果（本地回环，64B-1024B 均值）

协议	平均吞吐量 (msg/s)	平均端到端延迟 (μ s)
GMCP-R	65,931	14.96
Hash Chain	260,437	3.45
Seq+MAC	213,051	4.26
Ticket Only	364,572	2.29

数据来源：results/performance/performance_benchmark_results.csv。

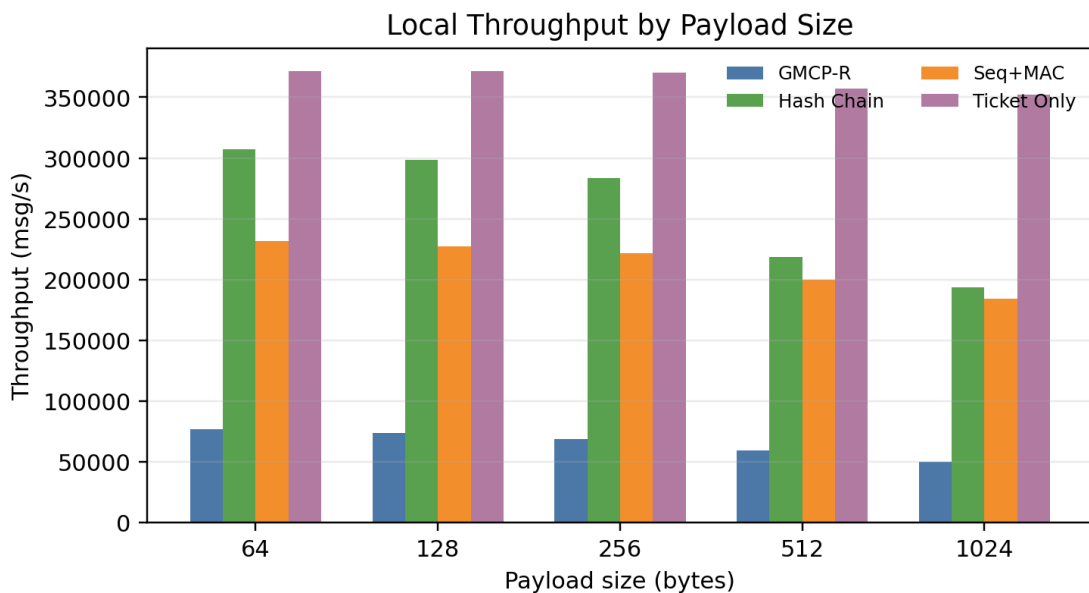


图 10. 本地纯计算吞吐量随载荷大小变化。

数据来源：results/performance/performance_benchmark_results.csv。

性能结果体现了 GMCP-R 的主要权衡：协议用额外计算开销换取历史连续性验证和恢复状态绑定。相较于 Ticket Only，GMCP-R 每条消息需要计算 payload_hash、更新 memory state，并对更多上下文字段进行认证；相较于单纯 Hash Chain，GMCP-R 还维护恢复票据和 checkpoint 相关状态。因此，GMCP-R 的吞吐量低于轻量基线，但其开销对应的是恢复后历史状态是否可信这一额外安全目标。

7.6 Concurrent Client Evaluation

并发实验测试 1、2、5、10 与 20 个客户端同时发送消息的情形。所有并发级别下，GMCP-R 均保持 100% 成功率，说明在当前测试规模内，协议的状态维护与服务器处理逻辑具有较好的稳定性。

表 13. 并发客户端测试结果

并发客户端数	成功率	总吞吐量 (msg/s)	平均 RTT (ms)	RTT 95% CI (ms)
1	100%	7,037	0.125	±0.007
2	100%	14,204	0.124	±0.004
5	100%	17,927	0.255	±0.007
10	100%	16,452	0.571	±0.015
20	100%	15,645	1.216	±0.018

数据来源：results/concurrent/concurrent_results.csv。

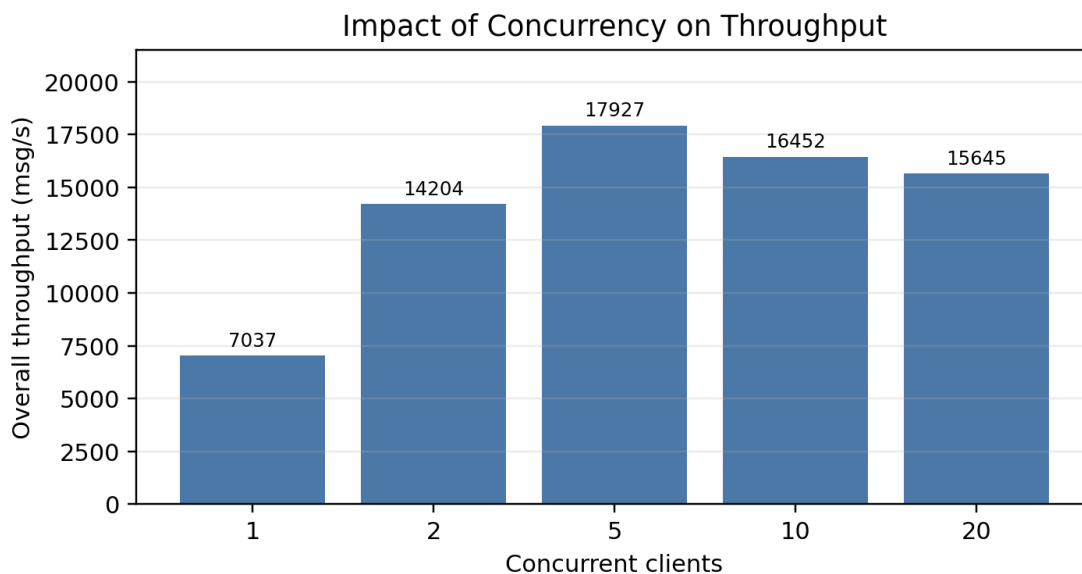


图 11. 并发客户端数量与总吞吐量变化。

数据来源：results/concurrent/concurrent_results.csv。

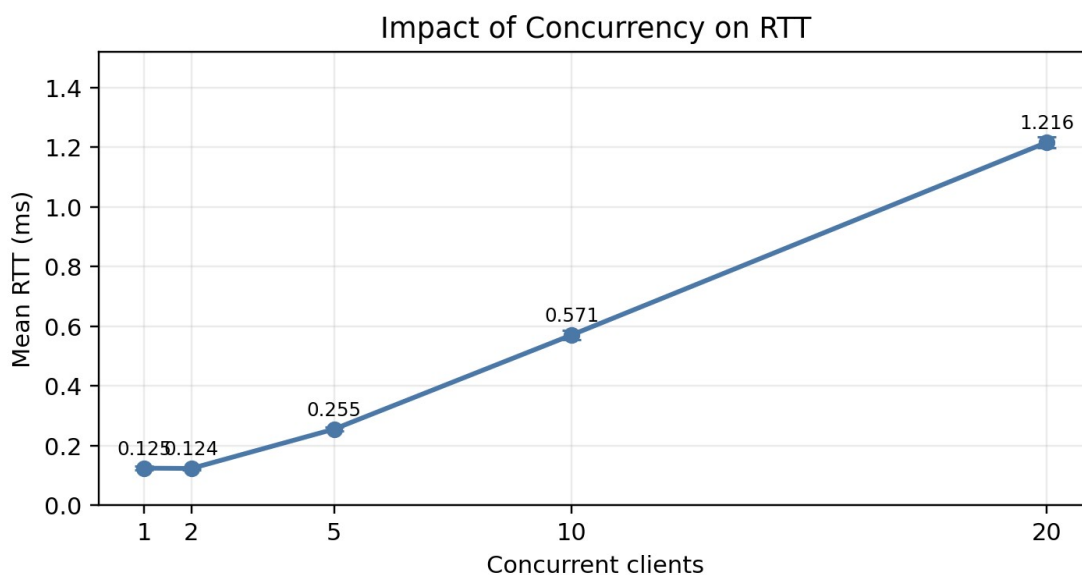


图 12. 并发客户端数量与平均 RTT 变化。

数据来源：results/concurrent/concurrent_results.csv。

并发结果显示，总吞吐量从 1 个客户端的 7,037 msg/s 增至 5 个客户端时的峰值 17,927 msg/s，随后在 10 和 20 个客户端下略有下降。该趋势与 Python 原型中的线程调度、GIL 影响、socket I/O 竞争以及服务端队列等待有关。平均 RTT 从 0.125 ms 上升到 1.216 ms，也反映了并发请求增加后的排队延迟和共享资源竞争。由于各并发级别成功率均为 100%，当前瓶颈主要表现为时延和吞吐变化，而非协议状态错误。

7.7 Preliminary Weak-Network Simulation

初步代码级弱网仿真用于观察当前实现中断连、丢包和恢复逻辑的行为。由于该实验尚未覆盖真实网络中的突发丢包、乱序、拥塞和带宽限制，本文不将其作为核心结论；相关表图仅作为补充实验列于 Appendix A，后续将使用 tc/netem 或 ns-3 进一步验证。

8. Discussion (讨论)

8.1 GMCP-R 实际解决的问题

GMCP-R 的目标不是替代 TLS/QUIC，也不是成为吞吐量最高的轻量认证协议，而是解决断续通信恢复后的历史状态可信性问题。实验中，GMCP-R 与 Hash Chain 均能检测历史断裂，但 GMCP-R 进一步通过 MemoryTicket 和 Checkpoint 给出可验证、可恢复且成本有界的恢复层。

8.2 恢复不只是重连

普通重连只说明双方重新获得了传输通道；GMCP-R 的恢复还要求 last_seq、last_mem 与 checkpoint 能够对应同一条合法历史链。因此，恢复成功意味着客户端可以从 last_seq + 1 接续发送新 DATA 报文，且接收端能够用恢复后的 last_mem 继续验证后续历史连续性。

8.3 安全性与性能的权衡

GMCP-R 的吞吐量低于 Ticket Only、Seq+MAC 和纯 Hash Chain，主要原因是每条消息需要同时计算 payload_hash、更新 memory state、覆盖更多上下文字段的 HMAC，并维护恢复票据和 checkpoint 状态。该额外开销换来的能力是恢复后历史状态是否可信，以及恢复成本是否能被 Checkpoint 限制。对于工业遥测、审计日志、金融流水和医疗记录等高价值场景，这一安全目标通常比单纯追求最高吞吐量更重要。

8.4 适用场景

GMCP-R 适合对历史连续性要求高、但又可能出现断连或弱网恢复的系统，如工业设备遥测、远程控制、审计日志同步、金融交易流水同步、医疗记录传输和边缘 IoT 数据上报。对于只需短连接请求响应的轻量系统，GMCP-R 的额外开销可能并非必要。

8.5 Resource Constraints in IoT and Edge Devices

当前实验主要评估吞吐量、延迟、并发成功率和恢复行为，但 IoT 与边缘设备还需要关注能耗、SRAM 占用和持久化存储开销。GMCP-R 的逐消息计算开销主要来自 payload hash、memory state update 和 HMAC；在线状态主要包括 last_seq、last_mem、最近 checkpoint 以及已消费 nonce 记录。与保存完整消息历史相比，GMCP-R 只保留紧凑的链式状态和快照信息，因此状态存储开销更小，更接近轻量级应用层协议的部署需求。

不过，本文尚未在真实 MCU 或嵌入式平台上测试能耗、SRAM 峰值和持久化写入频率。后续工作将考虑在 ARM Cortex-M、Raspberry Pi 或工业网关平台上复现实验，以评估哈希、HMAC、checkpoint 写入和 nonce 管理在资源受限设备上的实际成本。

8.6 Limitations (局限性)

1. 当前实现主要使用对称密钥和 HMAC，开放环境中还需要与 TLS/QUIC 密钥协商或非对称签名机制集成。
2. 实验主要为单服务器架构，分布式多副本场景下的记忆状态一致性和票据跨节点验证仍需研究。
3. 弱网实验目前为代码级仿真，后续应使用 tc/netem 或 ns-3 对丢包、乱序、抖动和带宽限制进行更接近真实网络的评估。
4. 当前攻击场景覆盖 drop、modify、replay、prev_mem 和五类无效票据，仍需扩展到协商降级、选择性篡改、并发恢复竞争等复杂攻击。

8.7 Future Work: Post-Quantum Key Establishment Integration

GMCP-R 当前依赖 HMAC-SHA256 和 SHA-256 来实现消息认证、票据认证和记忆链更新；这些机制并不等同于完整的后量子安全通信方案。面向长期安全通信，未来可以将 GMCP-R 与 ML-KEM/Kyber 或混合密钥交换机制结合，用后量子或混合密钥建立过程产生会话密钥，再由 GMCP-R 在应用层维护历史连续性和恢复状态。该方向属于后续集成研究，本文不声称已经实现后量子安全。

9. Conclusions (结论)

本文提出 GMCP-R，一种面向断续通信的历史连续性安全恢复协议。该协议通过 memory chain 绑定消息历史，通过 MemoryTicket 验证恢复请求，通过 Checkpoint 控制恢复开销；恢复过程恢复的是 last_seq、

last_mem 与 checkpoint 共同描述的历史记忆状态，而不是单纯恢复一条网络连接，从而缓解“连接恢复不等于历史状态恢复”的问题。

基于 Python TCP 原型的实验表明，在测试攻击场景下，GMCP-R 达到 100% 攻击检测率、0% 接受率与 0% 误拒绝率；五类无效 MemoryTicket 均被拒绝；本地纯计算吞吐量约为 65,931 msg/s；1 至 20 个并发客户端均保持 100% 成功率。初步代码级弱网仿真仅作为补充证据，真实弱网结论仍需进一步验证。后续工作将集中在形式化验证、真实弱网评估、嵌入式资源测量、后量子密钥建立集成和分布式多副本恢复机制上。

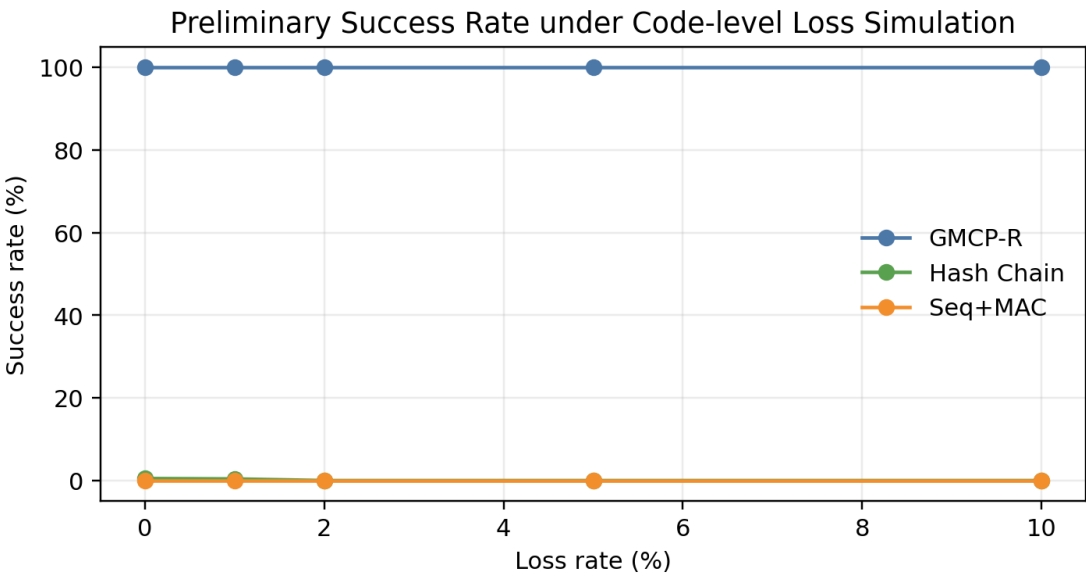
Appendix A. Weak-Network Supplement（补充弱网仿真）

本附录报告代码级弱网仿真的补充结果。该实验用于检查恢复机制在当前实现中的行为，不等同于操作系统级或网络仿真器级弱网评估，因此不作为本文核心结论。

附录表 A1. 弱网仿真初步结果（50 次重复均值）

协议	成功率均值	成功率标准差	平均吞吐量 (msg/s)	平均 RTT (ms)
GMCP-R	100.0%	0.0%	72.81	79.90
Hash Chain	0.18%	0.24%	0.00	23.77
Seq+MAC	0.0%	0.0%	0.00	0.00

数据来源：results/weak_network_simulation/summary_weak_network_simulation.csv。



附录图 A1. 丢包率变化下的协议成功率。

数据来源：results/weak_network_simulation/weak_network_simulation_results.csv。

